CSE 599k: LLM Serving Systems – Assignment 3

In this assignment, you will profile the performance of operations used in transformers and the transformer as a whole. The assignment consists of three parts: GEMM profiling, Attention Profiling and Transformer Profiling. Your submission will be in the form of a .zip file upload on Canvas, and must include a **short report with answers to the following questions (.pdf)** and the **code**.

- **Note**: When compiling libraries, please use make with "-j 256" rather than "-j" only. This would limit CPU usage and avoid crashing making the machine unresponsive.
- For both matrix multiplication and attention, we use FP16 data type.
- Use `-lcublas` with `nvcc` to compile

Section 1: General Matrix Multiplication

Q1. 2 + 5 + 3 points

- What is the operational intensity of a GEMM in terms of M, N, K dimensions?
    - **making an assumption that this is `[M, N] * [N, K] -> [M, K]`**

  Let `b` be the number of bytes of our data-type

  - We read `bN(M + K)` bytes of input, and write `bMK` bytes of output
  - We perform `MK` dot products on vectors of length `N`, each of which take `2N` FLOPS, for a **total of `2MNK` FLOPS**

$$\textbf{Operational Intensity} = \frac{2MNK}{b(KM+KN+MN)} \textbf{ FLOP/Byte}$$

- Calculate the compute intensity for M in 128 - 2048 with step 128, (N, K) = (512, 512), (4096, 4096), (14336, 4096), (4096, 1024), (1024, 4096) and show the result in a table. You can use https://www.tablesgenerator.com to generate tables.

  Values are in FLOP/Byte and data is assumed to be float16

M	(512, 512)	(4096, 4096)	(14336, 4096)	(4096, 1024)	(1024, 4096)
128	85.3333	120.4706	123.0558	110.7027	110.7027
256	128.0000	227.5556	236.9587	195.0476	195.0476
384	153.6000	323.3684	342.6932	261.4468	261.4468
512	170.6667	409.6000	441.1077	315.0769	315.0769
640	182.8571	487.6190	532.9368	359.2982	359.2982
768	192.0000	558.5455	618.8201	396.3871	396.3871
896	199.1111	623.3043	699.3171	427.9403	427.9403
1024	204.8000	682.6667	774.9189	455.1111	455.1111
1152	209.4545	737.2800	846.0590	478.7532	478.7532
1280	213.3333	787.6923	913.1210	499.5122	499.5122
1408	216.6154	834.3704	976.4458	517.8851	517.8851
1536	219.4286	877.7143	1036.3373	534.2609	534.2609
1664	221.8667	918.0690	1093.0674	548.9485	548.9485

M	(512, 512)	(4096, 4096)	(14336, 4096)	(4096, 1024)	(1024, 4096)
1792	224.0000	955.7333	1146.8800	562.1961	562.1961
1920	225.8824	990.9677	1197.9944	574.2056	574.2056
2048	227.5556	1024.0000	1246.6087	585.1429	585.1429

- What is CUTLASS? What is CUBLAS? What is the key difference between them?
    - https://docs.nvidia.com/cuda/cublas/
    - https://github.com/NVIDIA/cutlass

  CUBLAS is a closed-source library implementation of basic linear algebra operations that provides pre-built, optimized kernels/functions using well-rounded heuristics to perform well on a wide variety of workloads. On the other hand, CUTLASS is an open-source collection of C++ templates and tools that lets developers use basic kernels as building blocks/starting points for custom use-cases, letting them only use what they need. In some cases, CUBLAS will even use kernels instantiated from CUTLAS, though CUTLAS allows users to control exactly what is used whereas CUBLAS comes pre-built, operating sort of as a black box.

Q2. 5 + 5 + 3 + 2 points

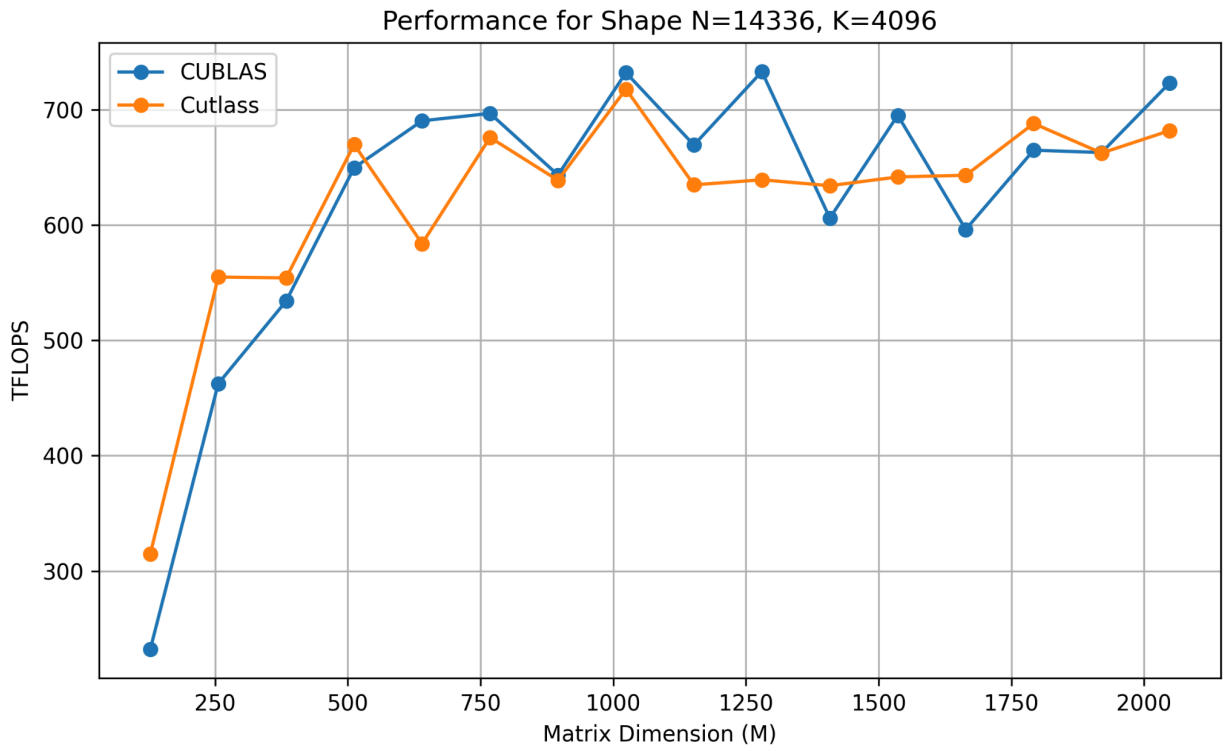
- Profile the CUBLAS cublasGEMMEx performance for the above shapes and plot the performance. Plot one figure for one shape with the M dimension on the x-axis and TFLOPs (total compute/time) on the y-axis.
    - Please use the profiling methods in `Section1/prof_example.cu`. We do warmups and L2 cache flush to increase the profiling accuracy.
    - The example plotting script is given at `Section1/plot_gemm.py.`

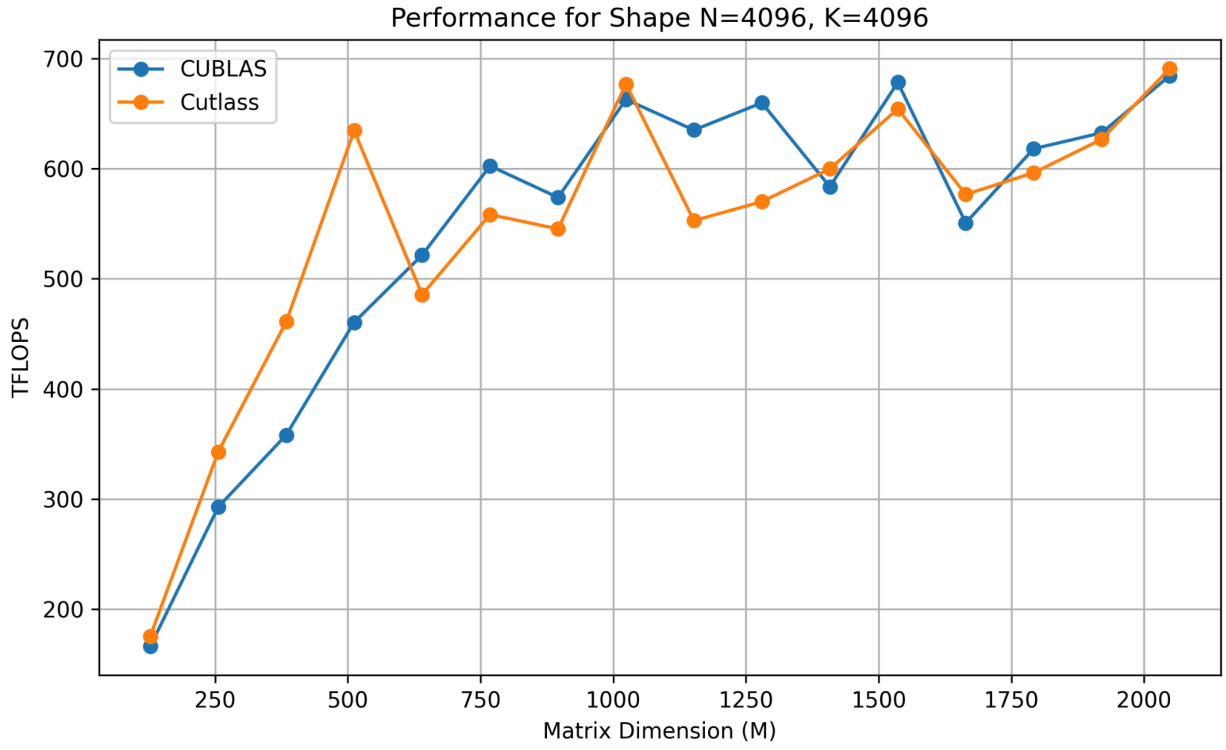
  See below for graphs

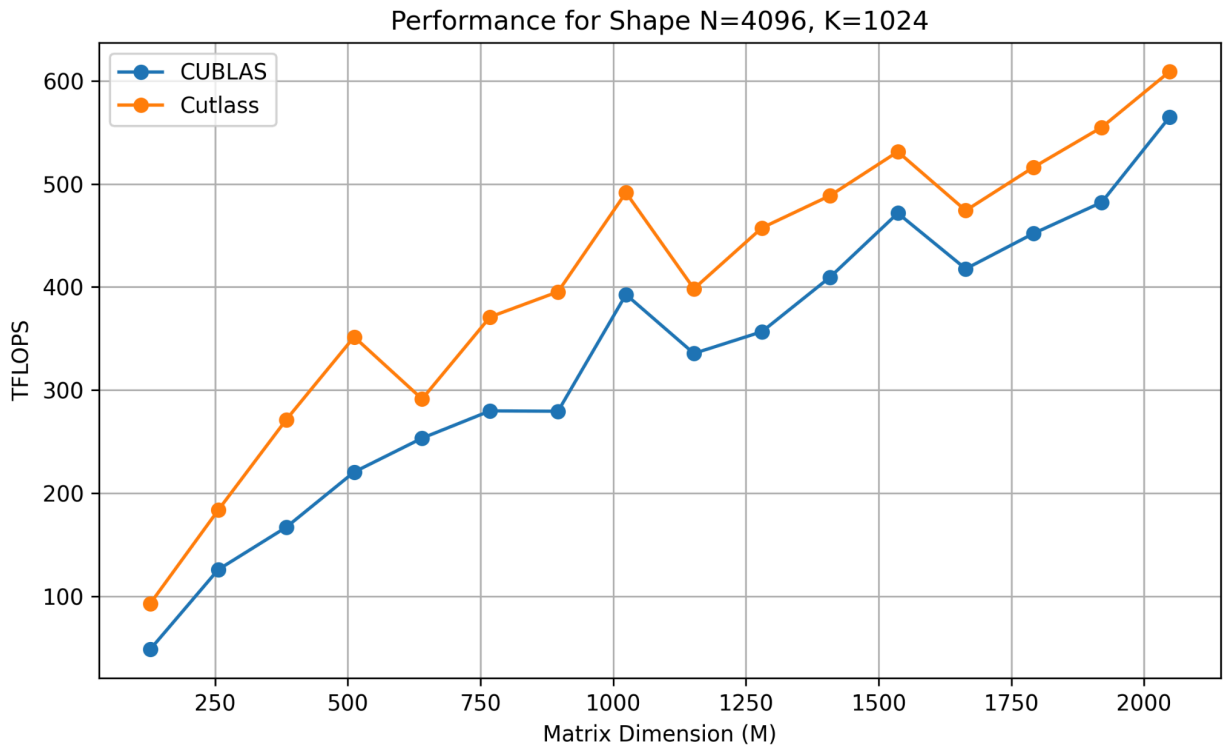
- Profile the CUTLASS GEMM performance using the CUTLASS profiler for the above shapes. The cutlass profiler will iterate through all kernel implementations with different tile sizes and configurations. Use again the `--profiling-iterations` of 100. Profile different --split_k_slices=1,2,4,8 with --split_k_mode=serial. Split K will control the number of splits in K dimension. For example, for GEMM shape (128, 1024, 4096), when split K is 4, the kernel will first treat it as 4 GEMM problems of size (128, 1024, 1024) and perform

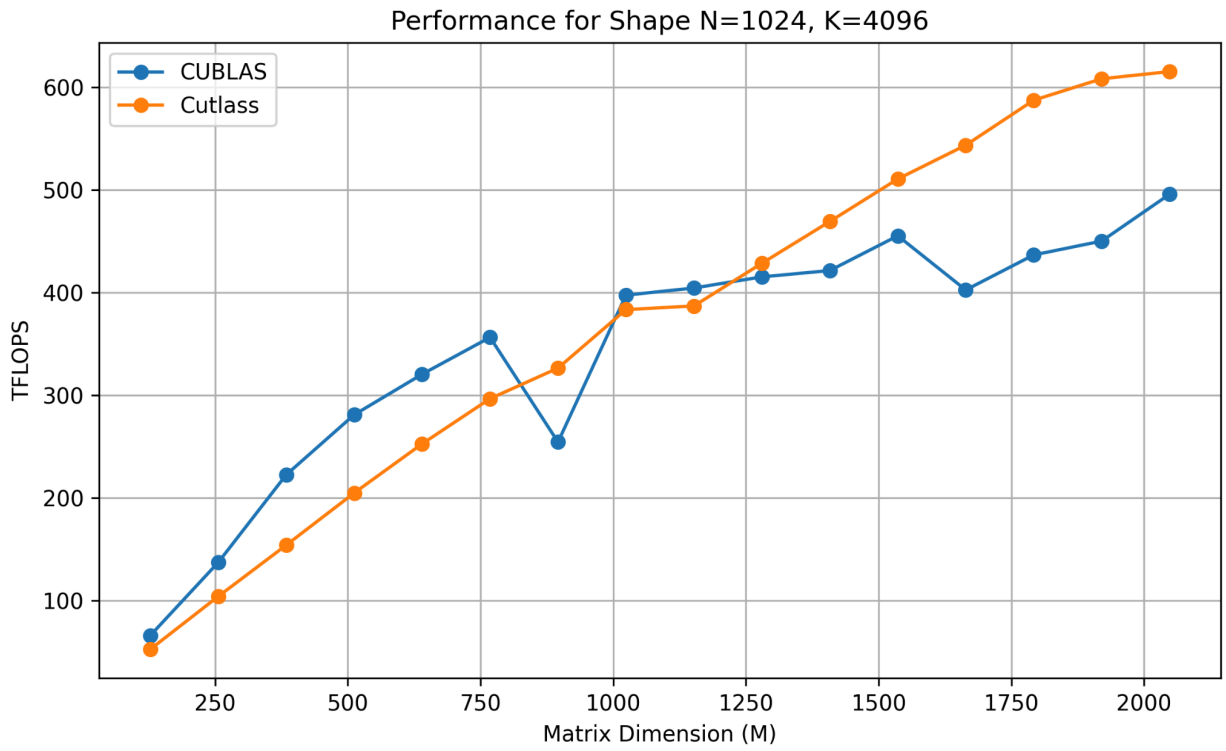
a reduction to add the partial results. Take the best performance across all implementations for each GEMM shape. Plot the CUTLASS performance in the same figure as CUBLAS.

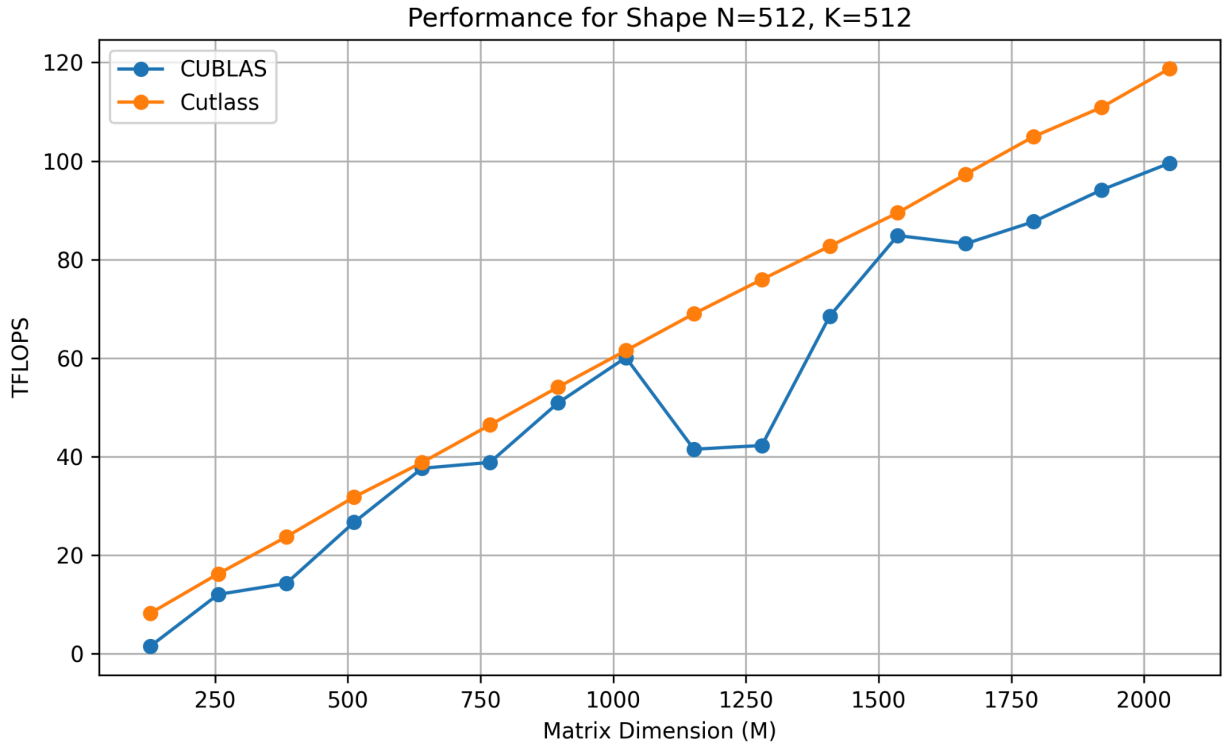
- https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/quickstart.md
- `cmake .. -DCUTLASS_LIBRARY_OPERATIONS=gemm -DCUTLASS_NVCC_ARCHS=90a -DCUTLASS_ENABLE_TESTS=OFF -DCUTLASS_UNITY_BUILD_ENABLED=ON`
- `make -j 256`

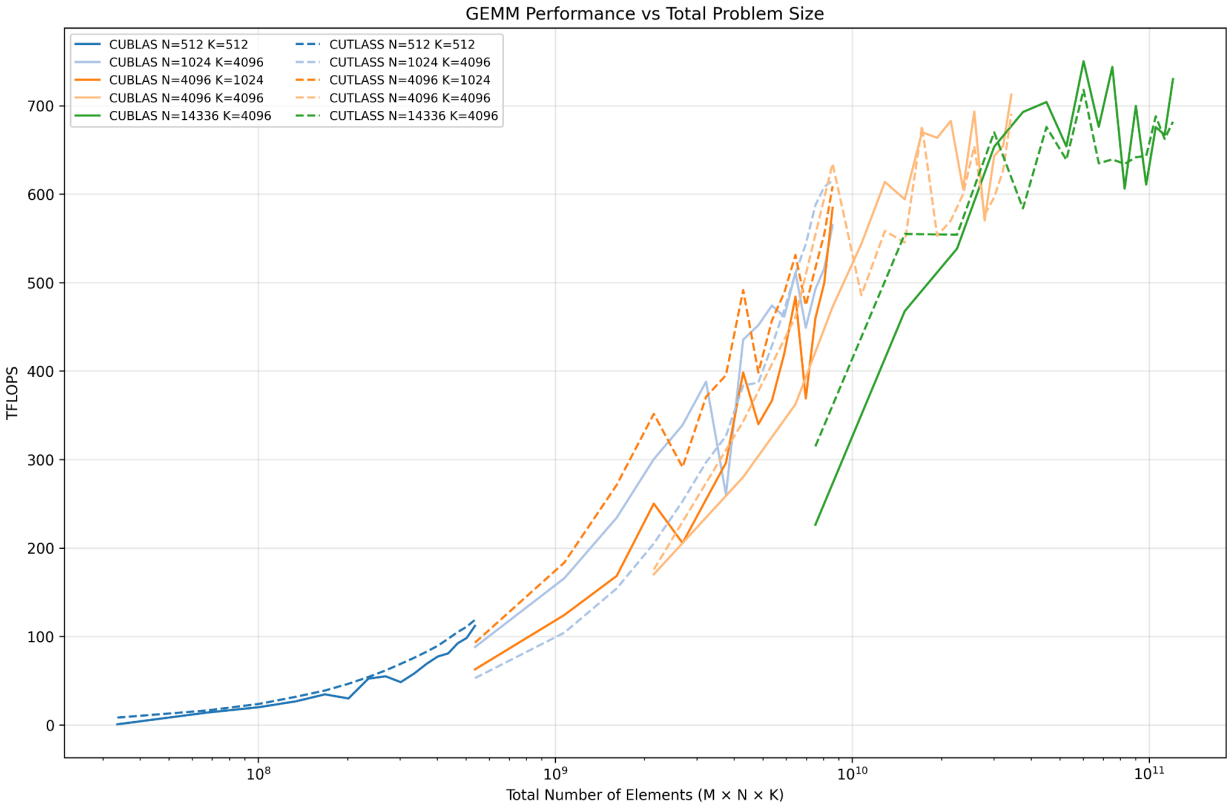

Performance for Shape N=14336, K=4096
CUBLAS
Cutlass
TFLOPS
300
400
500
600
700
250
500
750
1000
1250
1500
1750
2000
Matrix Dimension (M)


Performance for Shape N=4096, K=4096
CUBLAS
Cutlass
TFLOPS
200
300
400
500
600
700
250
500
750
1000
1250
1500
1750
2000
Matrix Dimension (M)


Performance for Shape N=4096, K=1024
CUBLAS
Cutlass
TFLOPS
100
200
300
400
500
600
250
500
750
1000
1250
1500
1750
2000
Matrix Dimension (M)


Performance for Shape N=1024, K=4096
CUBLAS
Cutlass
TFLOPS
100
200
300
400
500
600
250
500
750
1000
1250
1500
1750
2000
Matrix Dimension (M)


Performance for Shape N=512, K=512
CUBLAS
Cutlass
TFLOPS
0
20
40
60
80
100
120
250
500
750
1000
1250
1500
1750
2000
Matrix Dimension (M)



- Investigate the kernel that achieves the best performance in CUTLASS. What is the relationship between problem size, tile size, and split K?
    - https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/gemm_api_3x.md

  The fastest CUTLASS kernels always use a 128 x128x 64 tile size, which is the chunk of the matrix that fits the H100's hardware best (this seems to be true because of the way that GMMA instructions use the tensor cores: https://llvm.org/devmtg/2024-03/slides/nvidia-hopper-in-mlir.pdf). In particular, each thread block processes a 128x128 tile of the output matrix, batching 64 elements at a time (to the best of our understanding from the linked documentation which is very difficult to understand).

  If there aren't enough of these tiles (this depends on problem size) to keep all GPU cores busy, we slice the K dimension (split-K) just enough to create more work and fill the cores. This means that the K dimension is split into `split_k` "chunks", each of which are processed on a separate block. Any larger split-K just adds extra merge overhead (particularly synchronization overhead with serial mode, since it uses semaphores to order the reduction), so speed stops improving and can even decrease. As such we found that the split k of 2-4 was best for the GEMMs that we ran.

- What is the difference between the CUTLASS performance and the CUBLAS performance for different shapes? Describe your findings in a few sentences.

  CUTLASS marginally outperformed cuBLAS on the small 512 x 512 case and pulls noticeably ahead on the mid-sized 1024 x 4096 and 4096 x 1024/4096 shapes, where

the split-K options let it create enough extra parallel work to get slightly better performance. Note that in the smaller M cases the 1024 x 4096 GEMM has cuBLAS beating out CUTLASS likely because the optimizations present in CUTLASS aren't relevant at that specific size (and split-K doesn't help significantly)/ Once the problem size is much bigger (14336 x 4096 or 4096 x 4096 with higher M) both libraries already fully utilize the the tensor cores, so they have comparable performance (TFLOPs). The catch with CUTLASS is that it only has an advantage because we were looking at the top kernel (many of the other kernels are worse than cuBLAS) so the performance gain comes at the cost of extra tuning effort (not as generalizable) and less out-of-the-box reliability.

Section 2: Attention

Q1. 5 + 5 points

- Given model num_kv_heads, num_qo_heads, head_dim, what is the operational intensity of
    - **`b` = number of bytes of our data-type (2)**
    - **`H_qo` = number of query heads**
    - **`H_kv` = number of key/value heads**
    - **`d` = head dimension**
    - Prefill attention, when the prompt length is p
        - We first perform a matrix multiplication **for each qo head** multiplying Q by K^T which is going to be `H_qo*2*d*p^2` FLOPs. Then we perform softmax on the resulting p x p matrix resulting in `H_qo*3*p^2` FLOPs. Finally, we multiply the p x p matrices by V resulting in `H_qo*2*d*p^2` FLOPs. This results in a total of `H_qo*4*d*p^2 + H_qo*3*p^2` FLOPs.
        - We read the Q, K, and V values one time each of which have `p(d*H_qo + 2*d*H_kv)` number of elements and we write O one time which has `p*d*H_qo` number of elements so total this results in `b*p*(2*d*H_qo + 2*d*H_kv)` bytes operated on
        - Operational Intensity: `(H_qo*4*d*p^2 + H_qo*3*p^2)/ b*p*(2*d*H_qo + 2*d*H_kv) = (H_qo*4*d*p + H_qo*3*p)/ b(2*d*H_qo + 2*d*H_kv)`

          $$\frac{H_{qo}p(4d+3)}{2bd(H_{qo}+H_{kv})} = \frac{p(4d+3)}{2bd} \cdot \frac{H_{qo}}{(H_{qo}+H_{kv})}$$
    - Decode attention, when the context length is c

- We first perform a matrix multiplication **for each qo head** multiplying Q by K^T which is going to be `H_qo*2*1*d*c` FLOPs. Then we perform softmax on the resulting 1 x c matrix resulting in `H_qo*3*c` FLOPs. Finally, we multiply these matrices by V resulting in `H_qo*2*d*c` FLOPs. This results in a total of `H_qo*4*d*c + H_qo*3*c` FLOPs.
- We read the Q, K, and V values one time with Q being only 1 token long (assuming KV Cache) and K/V being the complete context length (c) long so this is `d*H_qo + 2*d*c*H_kv` number of elements and we Write O one time (note that this again only has one token since we only care about the final token in KV cache case) which is `d*H_qo` number of elements. In total this is `b*d*2(H_qo + c*H_kv)` bytes operated on.
- Operational Intensity: `(H_qo*4*d*c + H_qo*3*c) / b*d*(2*H_qo + 2*c*H_kv)= (H_qo*c(4d + 3))/b*d*(2*H_qo + 2*c*H_kv)`
- $\frac{H_{qo}c(4d+3)}{2bd(H_{qo}+cH_{kv})} = \frac{c(4d+3)}{2bd} \cdot \frac{H_{qo}}{(H_{qo}+cH_{kv})}$

- Compute the operational intensity for llama2-7B, llama3-8B, llama3-70B

Model	**H_qo**	**H_kv**	**d**
LLaMA2-7B	32	32	128
LLaMA3-8B	32	8	128
LLaMA3-70B	64	8	128

  - For prefill attention, p = 2 ^ 7 - 2 ^ 15

Sequence Length (p)	**LLaMA2-7B**	**LLaMA3-8B**	**LLaMA3-70B**
128	64.3750	103.0000	114.4444
256	128.7500	206.0000	228.8889

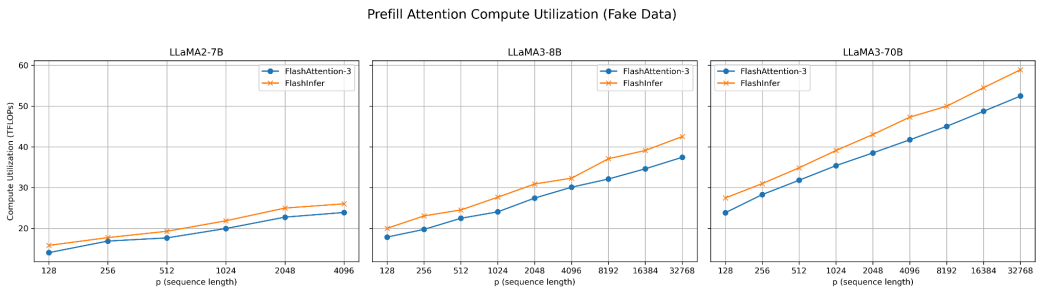
Sequence Length (p)	LLaMA2-7B	LLaMA3-8B	LLaMA3-70B
512	257.5000	412.0000	457.7778
1024	515.0000	824.0000	915.5556
2048	1030.0000	1648.0000	1831.1111
4096	2060.0000	3296.0000	3662.2222
8192	4120.0000	6592.0000	7324.4444
16384	8240.0000	13184.0000	14648.8889
32768	16480.0000	26368.0000	29297.7778

- For decode attention, c = 2 ^ 7 - 2 ^ 15

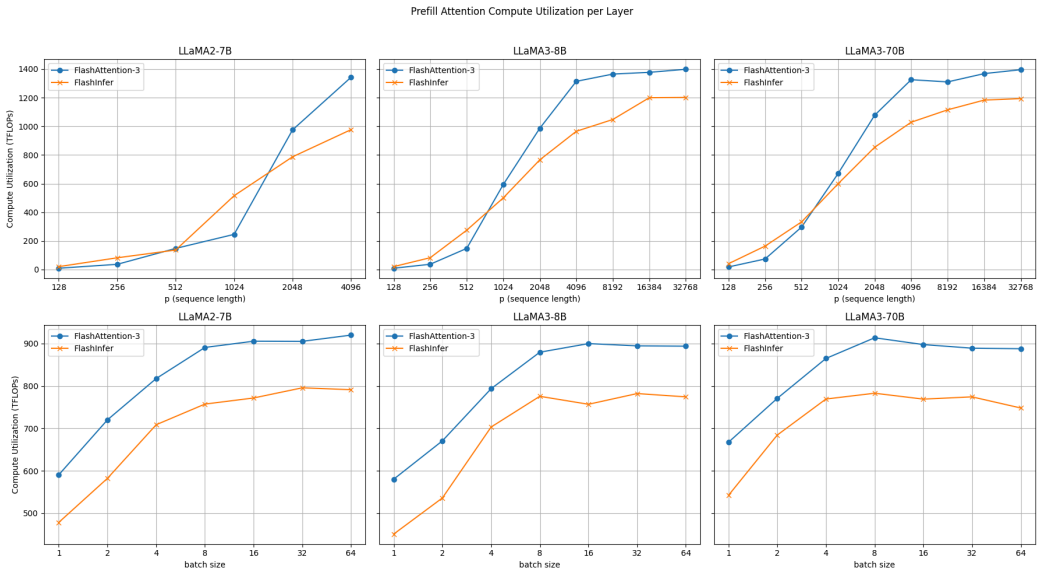
Context Length (c)	LLaMA2-7B	LLaMA3-8B	LLaMA3-70B
128	0.998062	3.901515	7.573529
256	1.001946	3.961538	7.803030
512	1.003899	3.992248	7.923077
1024	1.004878	4.007782	7.984496

Context Length (c)	LLaMA2-7B	LLaMA3-8B	LLaMA3-70B
2048	1.005368	4.015595	8.015564
4096	1.005614	4.019512	8.031189
8192	1.005737	4.021474	8.039024
16384	1.005798	4.022455	8.042948
32768	1.005829	4.022946	8.044911

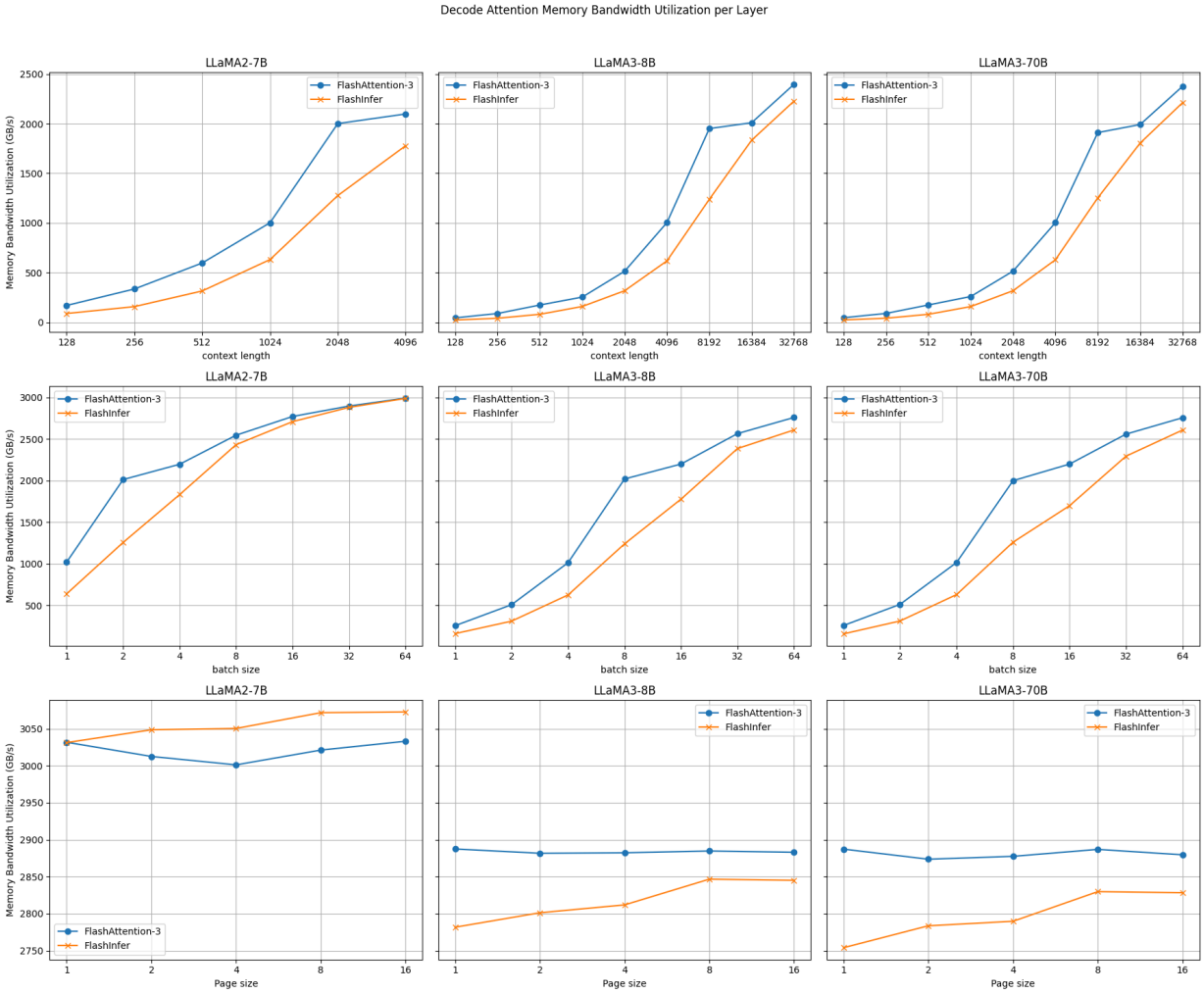
Q2. 10 + 10 points

- Install Flashinfer using `uv pip install flashinfer-python==0.2.4 -i https://flashinfer.ai/whl/cu124/torch2.6/`.
- Install FlashAttention 3 by cloning the repo https://github.com/Dao-AILab/flash-attention and run `` `cd hopper && python setup.py install` ``
- Evaluate the prefill attention performance of one layer using flash_attn3 https://github.com/Dao-AILab/flash-attention and FlashInfer https://github.com/flashinfer-ai/flashinfer/ for
    - Batch size = 1, p = 2 ^ 7 - 2 ^ 12 for llama2-7B, p = 2 ^ 7 - 2 ^ 15 for llama3-8B and llama3-70B. Plot the compute utilization TFlops of each model. Use **$\log_2(p)$** as the x-axis. Generate one **subplot per model** inside a shared figure. In each subplot, draw line plots for both FlashAttention-3 and FlashInfer using different line styles or colors. Label axes clearly. (see `Section2/plot_attention_p.py`)
    - 

- Batch size = 2 ^ 0 - 2 ^ 6, p = 1024 for all three models. Plot compute utilization using log batch size as the x-axis.



- Evaluate the decode attention performance of one layer with paged attention using flash_attn3 and Flashinfer
    - Batch size = 1, page size = 16, c = 2 ^ 7 - 2 ^ 12 for llama2-7B, p = 2 ^ 7 - 2 ^ 15 for llama3-8B and llama3-70B. Plot memory bandwidth utilization (GB/s) similar to previous questions.
    - Batch size = 2 ^ 0 - 2 ^ 6, c = 1024 for all three models. Plot memory bandwidth utilization similarly.
    - Batch size = 128, c = 1024, page size = 1, 2, 4, 8, 16 for all three models. Plot the memory bandwidth utilization of each model.



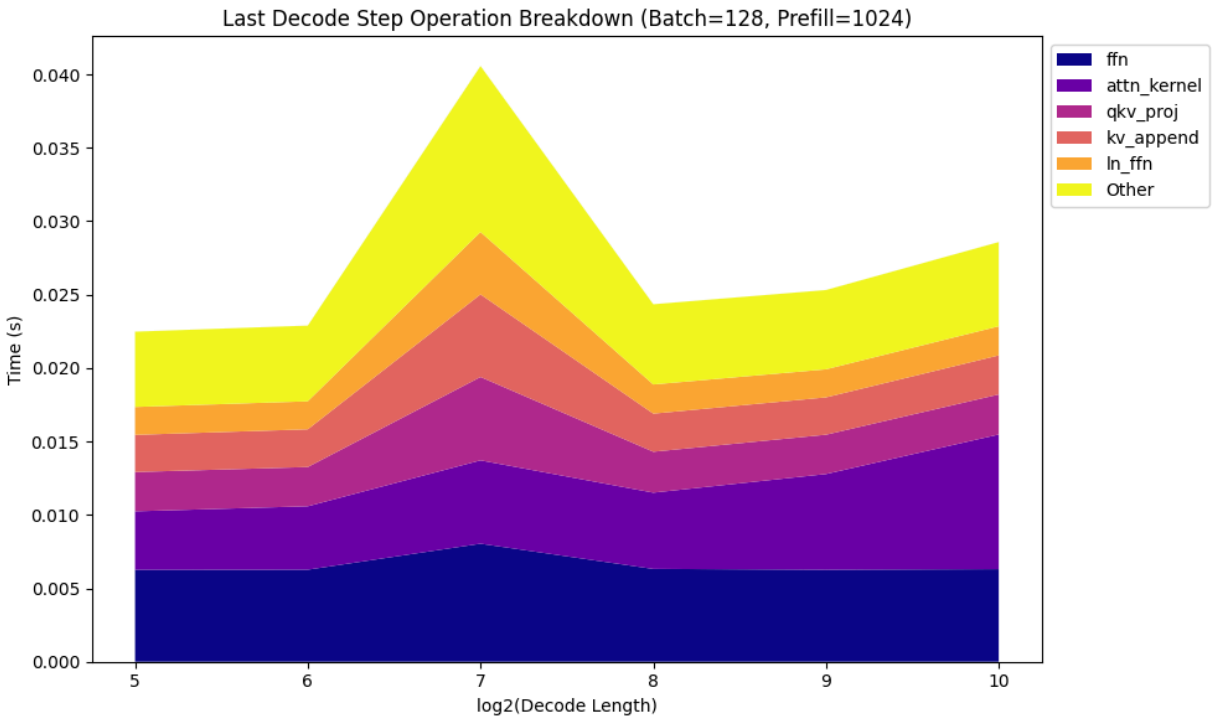
Section 3: End-to-end performance

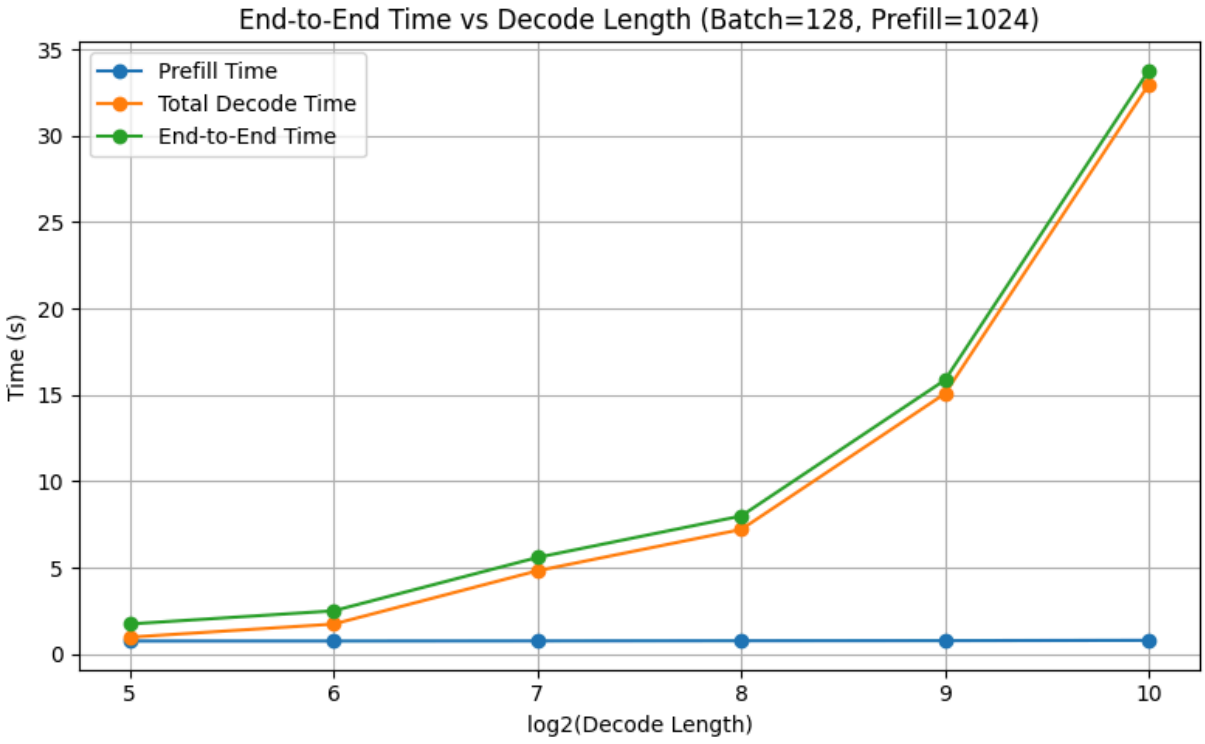
Q1. 15 points

- Starting from the `Section3/flashinfer_pipeline.py`, construct a serving engine using FlashInfer attention and rope kernels with paged attention.

Q2. 10 + 10 + 10 points

- With batch size 128, prefill length 1024, decode length 2^5 - 2^10, profile the prefill time and total decode time. Plot end-to-end time curve with the log(decode length) as the x-axis. Which phase is the key bottleneck? In the last decode cycle, which operation takes the longest time for different decode lengths?



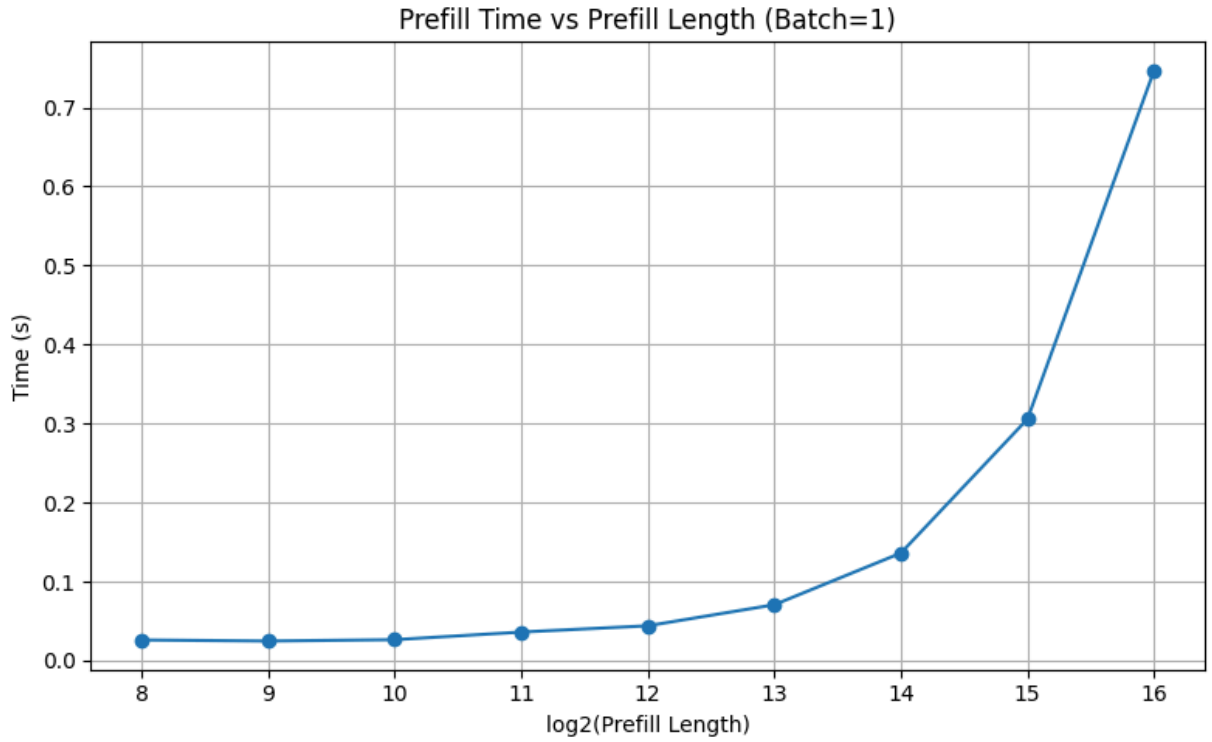


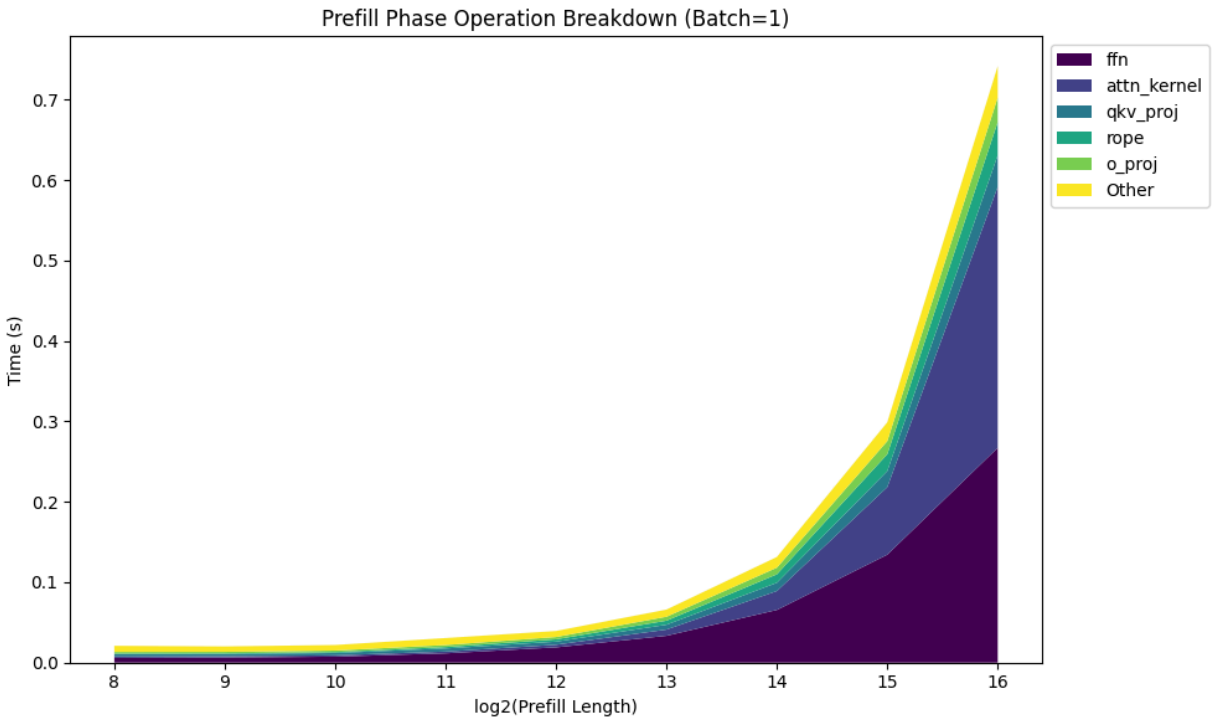
It's immediately clear that the decode phase is the bottleneck. The prefill stage incurs a fixed overhead of less than 2 s, while decode time grows roughly linearly with the number of output tokens. That matches our intuition: prefill processes the

input in one pass, but each new token in decode triggers another full attention pass over all prior tokens, so work accumulates linearly as the sequence length increases. Additionally, we do not change the number of prefill tokens so the amount of time prefill takes not changing is unsurprising.

Across the range of decode lengths, the dominant last-decode operation shifts; at short lengths (2^6 tokens) the FFN is slowest, while as decode length grows further, the attention kernel becomes the clear bottleneck, rising steadily to dominate at 2^8 tokens. This pattern reflects how the FFN isn't dependent on the decode length (during the decode phase we only pass one token through mlp which means that we will always do the same amount of computation) whereas the attention kernel is dependent on the decode length (the longer the length the more flops you do during the attention operation). In practice, this suggests that optimizing FFN kernels matters most for short sequences, reducing per-step overhead is critical around 128 tokens, and attention optimizations will yield the largest performance gains on long sequences.

- With batch size 1, prefill length 2^8 - 2^16, profile the prefill time. Plot end-to-end time curve with log(prefill length) as x-axis. Examine the prefill time breakdown. What operations are the dominating factor for various prefill lengths?

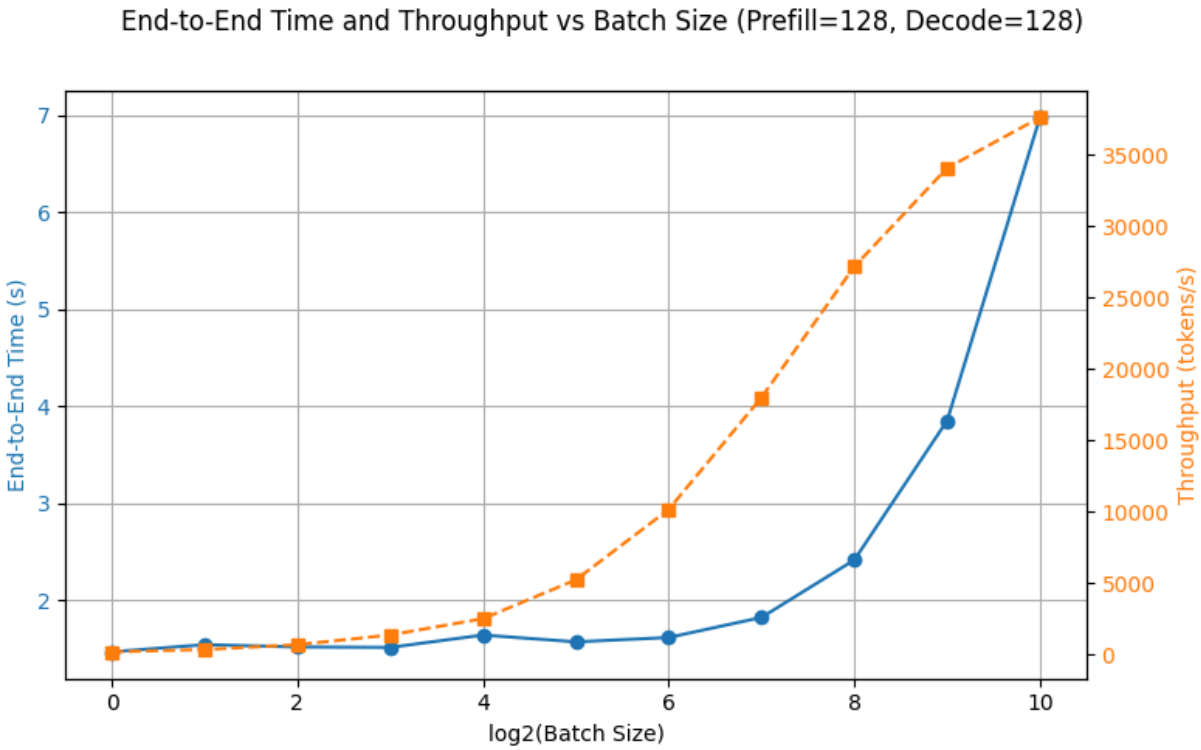




Throughout the prefill phase, the FFN operation dominates for sequence lengths up to around 2^15 tokens, where the attention operation's latency starts to dominate. This is expected; in attention each token attends to every other token (O(n^2)), whereas the

FFN works on each token independently (O(n)). The graph shows that self-attention latency finally overtakes FFN at around 2^16 tokens, illustrating that attention is cheap on short inputs but becomes the primary bottleneck for large sequence lengths. This can be confirmed from the graph because the ffn time grow ~linearly (approx. doubles with each doubling of prefill length) while the attn_kernel time grows faster than linearly (more than doubles with each doubling of prefill length).

- With batch size 2^0 - 2^10, prefill length 128, decode length 128, plot the end-to-end time curve with log(batch size) as x-axis. Plot the total throughput ((prefill + decode) / time) curve with log(batch size) as x-axis. When does the performance saturate?



We see that the performance begins to saturate at a batch size of around 2^9-2^10. We see a plateau at this point as the GPU is being nearly fully utilized without break at this point (due to additional overhead introduced from our profiling code this is slightly later than we may potentially expect but not unreasonable). As confirmed by course staff, we'd expect an earlier saturation point if this overhead was removed.